Marburg University

Lecture
Computer Vision
Chapter 3 –
Image Representation & Transformations

Prof. Dr. Ralph Ewerth
Research Group AI – Multimodal Modelling and Machine Learning
Department of Mathematics and Computer Science
Marburg University & Hessian Center for Artificial Intelligence (hessian.AI)

Content of the Lectures

Date	Lecture nr.	Topic
16.10.2025	1	Organisation; Introduction Computer Vision, Color spaces
30.10.2025	2	Linear filters,
06.11.2025	3	Image transformations (Fourier, DCT, Wavelets)
Tba	4	Introduction to (machine) learning for computer vision
Tba	5	Gradient-based learning and generalisation
27.11.2025	6	Neural networks
04.12.2025	7	Neural networks II
11.12.2025	8	Convolutional neural networks
18.12.2025	9	Transformer models
15.01.2026	10	Representation learning
22.01.2026	11	Generative models
29.01.2026	12	Pre-training and transfer learning
05.02.2026	13	Vision tasks and applications (Motion, Depth, Segmentation…)
12.02.2026	14	Vision and language

Literature for this Lecture

- W. Burger, M. J. Burge: „Digitale Bildverarbeitung“
  Springer-Verlag, 3. Auflage, 2015
  Chapter 18-20

- In English: „Digital Image Processing – An Algorithmic Introduction using Java“
  Springer-Verlag, 2016
  Chapter 18-20

- Tilo Strutz: Bilddatenkompression.
  Vieweg, 4. Auflage, 2009
  Chapter 6.2 (6.2.1-6.2.4, 6.2.6, 6.2.8; 6.3.1)

Image Representation - Recap

- Representation at pixel level does not represent well visual content

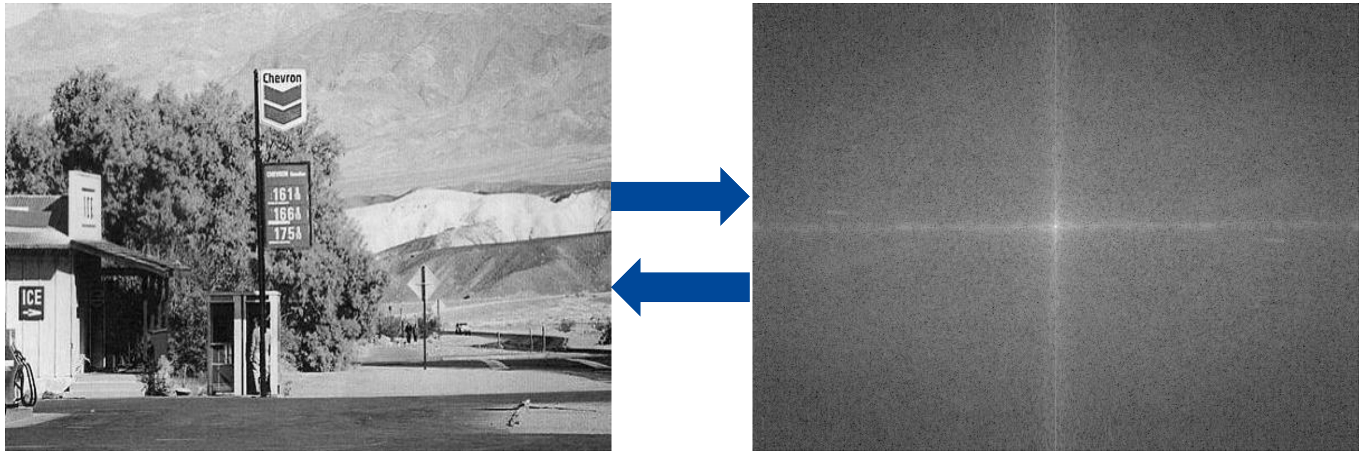
- Task: Find appropriate representations for
  - color
  - shape
  - texture
  - objects
  - regions

Fourier Transform

Fourier transform: Describe any function as
- the sum of weighted periodic functions (basis functions)
- with different frequencies (frequency domain representation)

Applications
- Compression
- Image analysis
- Fast filtering

Transformation must be **invertible**



Fourier Series Reloaded

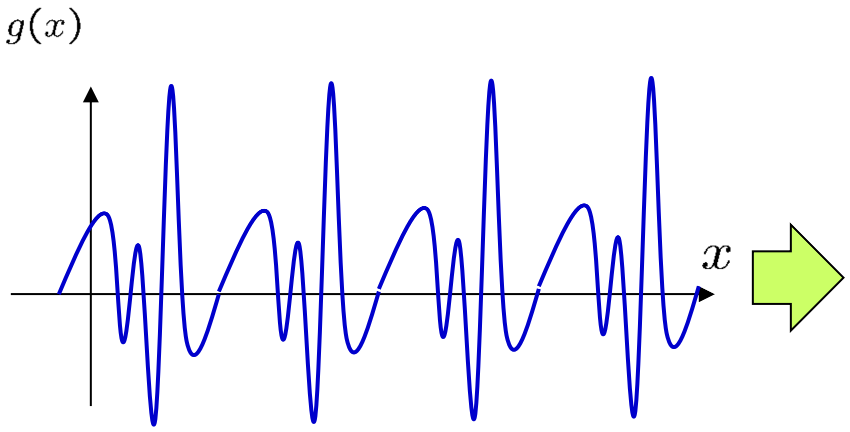
Every periodic function g(x)
with a base frequency ω_0
can be represented
by a generally infinite sum of harmonic (co)sine functions in the form

$$g(x) = \sum_{k=0}^{\infty} \left[A_k \cdot \cos(k\,\omega_0\,x) + B_k \cdot \sin(k\,\omega_0\,x) \right]$$

Cosine function (and sine function) with frequency $k\omega_0$ and amplitude A_k (B_k)

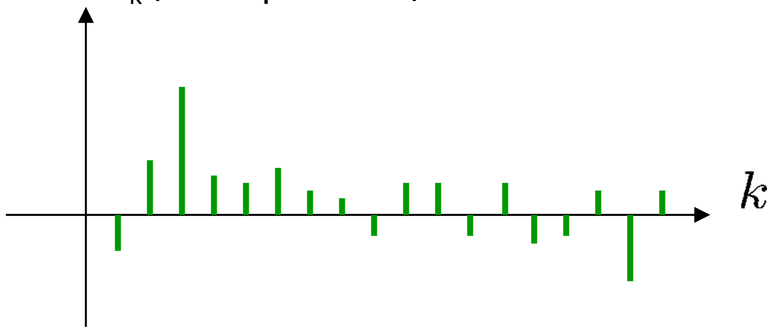
A Fourier series is therefore a sum of cosine and sine functions

Cosine and Sine Spectrum



A_k (cosine spectrum)

B_k (sine spectrum)



$$g(x) = \sum_{k=0}^{\infty} \left[A_k \cdot \cos(k\,\omega_0\,x) + B_k \cdot \sin(k\,\omega_0\,x) \right]$$

Fourier Integral

- Representation of **non-periodic** functions as a sum of cosine and sine functions with **Fourier integral**

$$g(x) = \int_0^\infty A_\omega \cos(\omega x) + B_\omega \sin(\omega x) \, \mathrm{d}\omega$$

- Is possible, but requires infinitely many, closely adjacent frequencies ω
- Determination of the coefficients

$$A_\omega = A(\omega) = \frac{1}{\pi} \int_{-\infty}^{\infty} g(x) \cdot \cos(\omega x) \, \mathrm{d}x$$

$$B_\omega = B(\omega) = \frac{1}{\pi} \int_{-\infty}^{\infty} g(x) \cdot \sin(\omega x) \, \mathrm{d}x$$

Fourier Transform

- Output signal and spectrum are complex-valued functions

- Starting from the functions A(ω) and B(ω) of the Fourier integral, the complex-valued Fourier spectrum G(ω) of a complex-valued function is defined as:

$$
\begin{aligned}
G(\omega) &= \sqrt{\tfrac{\pi}{2}}\Big[A(\omega) - \mathrm{i}\cdot B(\omega)\Big] \\
&= \sqrt{\tfrac{\pi}{2}}\left[\frac{1}{\pi}\int_{-\infty}^{\infty} g(x)\cdot\cos(\omega x)\,\mathrm{d}x \;-\; \mathrm{i}\cdot\frac{1}{\pi}\int_{-\infty}^{\infty} g(x)\cdot\sin(\omega x)\,\mathrm{d}x\right] \\
&= \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty} g(x)\cdot\Big[\cos(\omega x) - \mathrm{i}\cdot\sin(\omega x)\Big]\,\mathrm{d}x
\end{aligned}
$$

- Euler's notation for complex numbers:

$$
G(\omega) = \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty} g(x)\cdot e^{-\mathrm{i}\omega x}\,\mathrm{d}x
$$

Basis Functions

- Images can be understood as a two-dimensional function f(m,n)

- A function can be understood as a sum of N weighted basis functions b_u:

$$f(n) = \sum_{u=0}^{N-1} w_u \cdot b_u(n)$$

- The weights w_u form a new function w(u), which describes f(n) exactly together with the basis functions

- The effect of changes on the weights (e.g. filtering) depend on the basis functions.

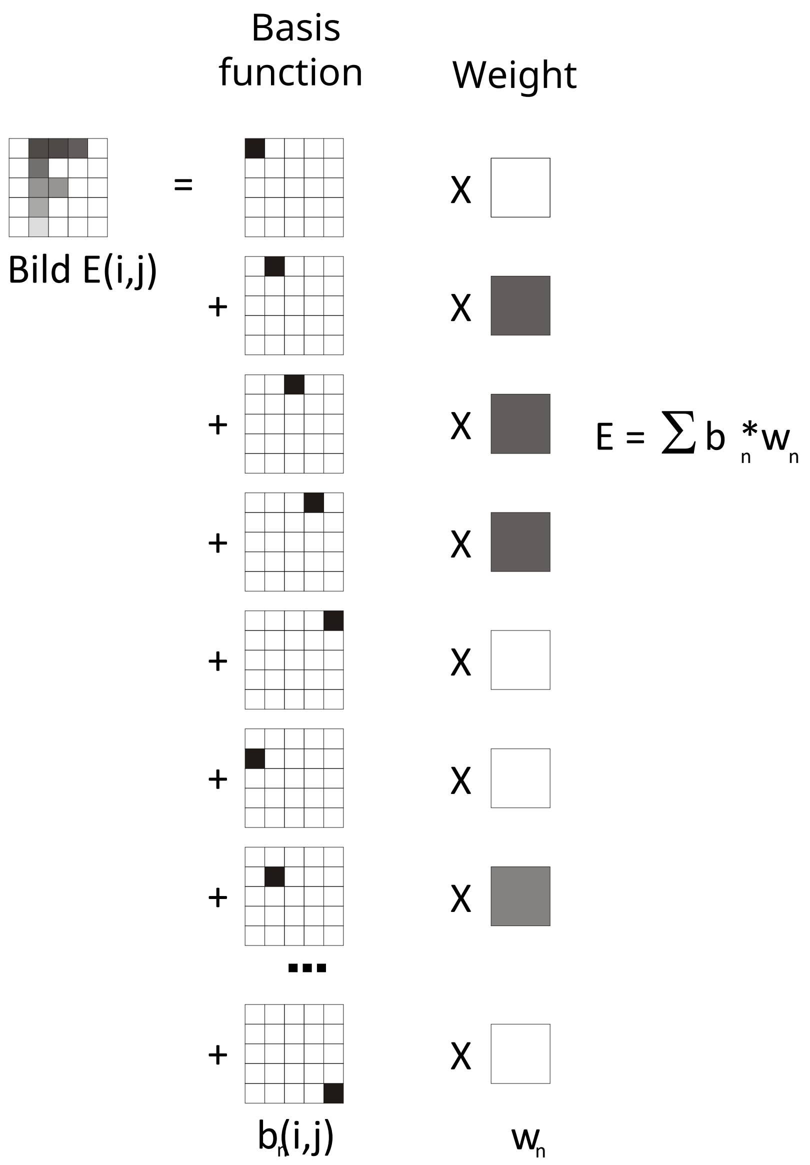
- The Fourier transformation is the transformation from
  - a spatial basis -> into a frequency basis.

Spatial Basis (Space Domain Basis)

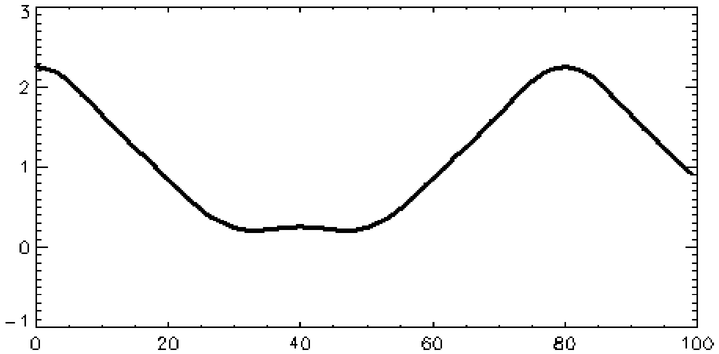
- Each basis function is an image that
  - has the value 1 in one pixel and
  - the value 0 in all other pixels.

- Two basis functions b_k and b_l differ in that they have the '1-pixel' at different positions.

- The proposed basis is
  - complete
  - unambiguous

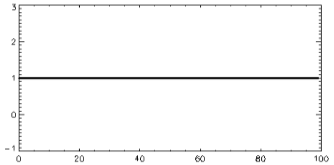


Fourier Transformation: Set of Basis Functions

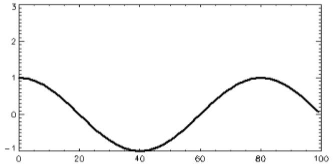


=

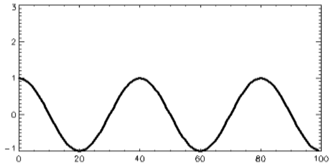
1.0·



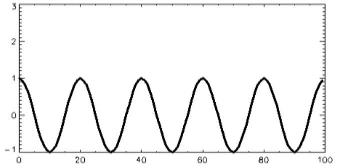
+1.0



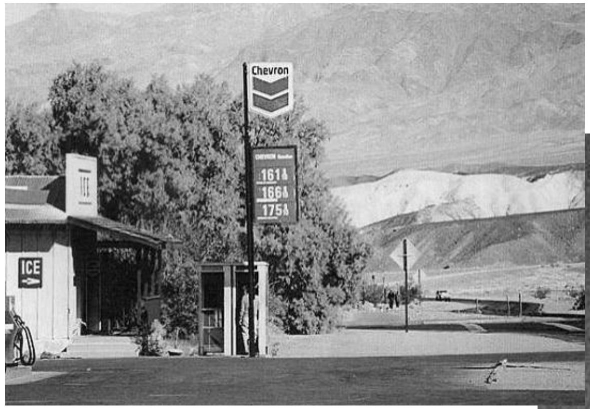
+0.2

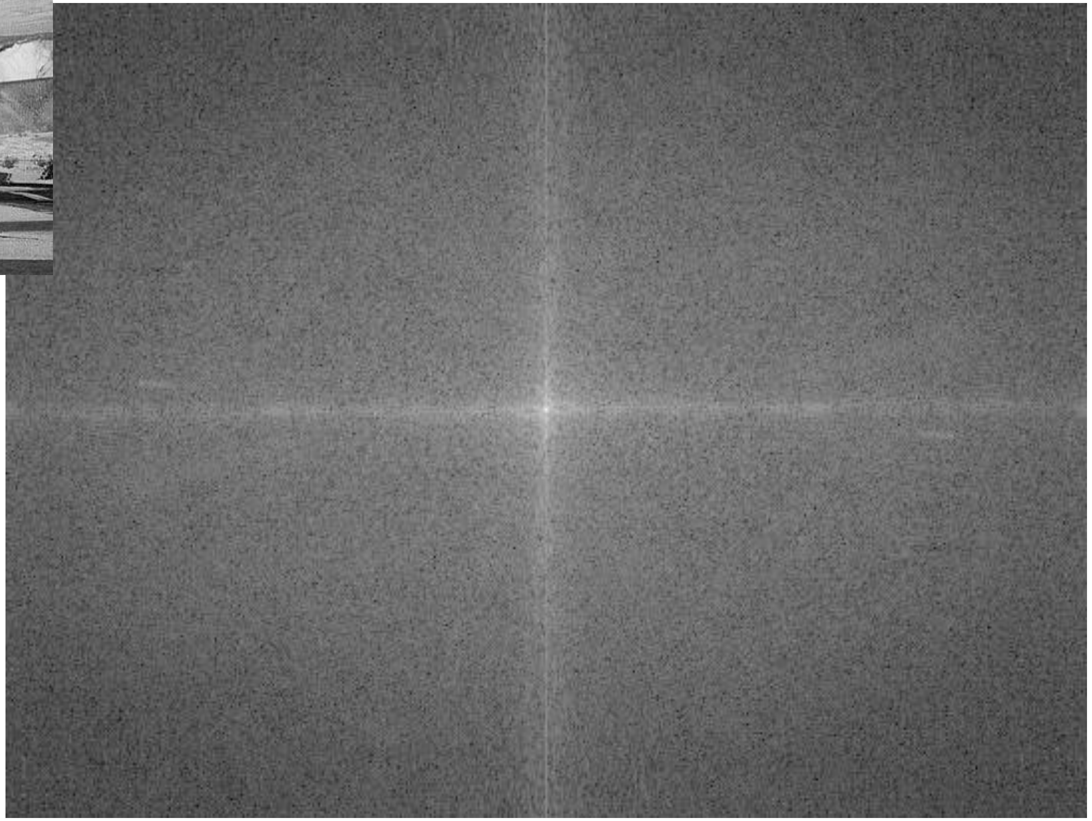


+0.05



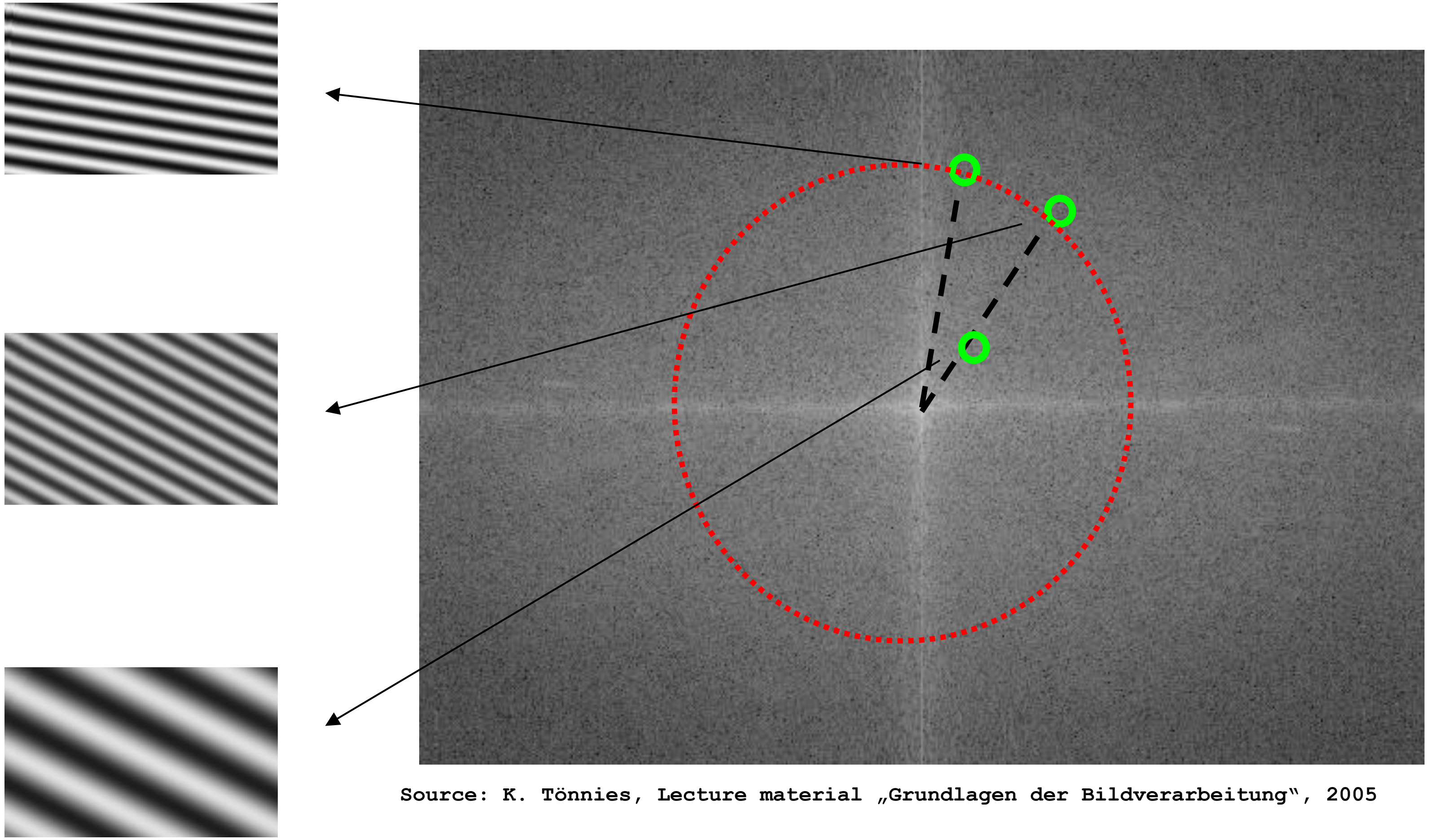
An Example in 2D





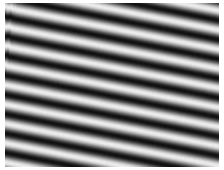
Source: K. Tönnies,
Lecture material
„Grundlagen der
Bildverarbeitung“, 2005

An Example in 2D

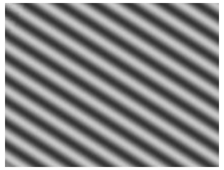


Source: K. Tönnies, Lecture material „Grundlagen der Bildverarbeitung", 2005

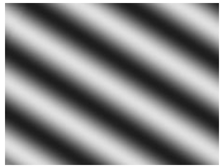
An Example...



+



+



+...=

Source: K. Tönnies, Lecture material „Grundlagen der Bildverarbeitung", 2005

Invertibility

- The transformation is invertible when the basis functions form an **orthogonal basis**
  - What is orthogonality for functions?
  - When do basis functions form an orthogonal basis?

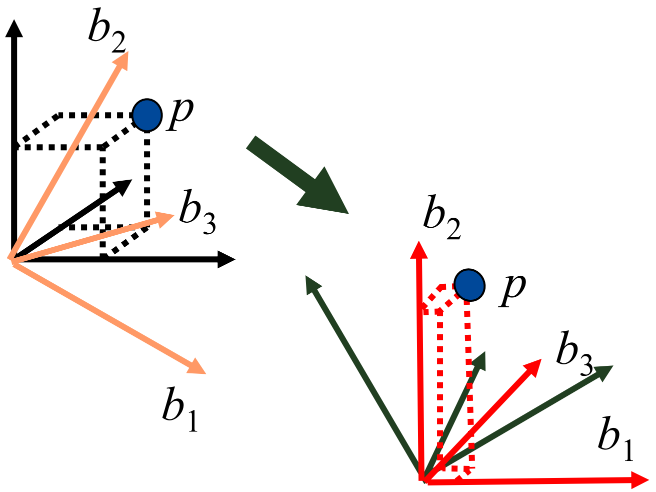
- Transformation: 'Projection' of the function **onto the new basis**

- Inverse transformation: Projection **onto the original basis**

- Orthogonality, projection, transformation etc. are concepts from **linear algebra** and can also be used for functions.

Orthogonal Vector Basis

- Two vectors are orthogonal when their scalar product is zero
- N vectors form a basis for an N-dimensional space when they are all orthogonal to each other.
- Projection of a vector ***p*** onto a basis vector ***b*** is given by the normalized scalar product between them.
- Example: Coordinate vectors of a 3D space.



$$\vec{b}_i \bullet \vec{b}_j = 0, \quad \text{für } i, j = 1,2,3 \wedge i \neq j$$

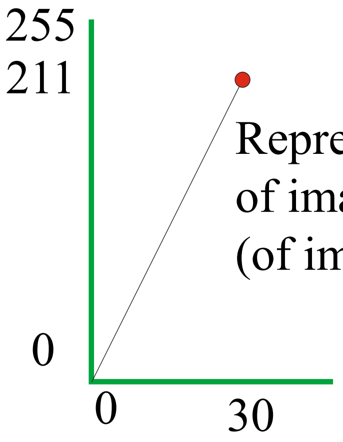
$$\vec{p}^{\,B} = \begin{pmatrix} \frac{\vec{p} \bullet \vec{b}_1}{\left\|\vec{b}_1\right\|} & \frac{\vec{p} \bullet \vec{b}_2}{\left\|\vec{b}_2\right\|} & \frac{\vec{p} \bullet \vec{b}_3}{\left\|\vec{b}_3\right\|} \end{pmatrix}$$

Functions instead of Vectors

- A function f(x) is also uniquely defined by complete enumeration of all values: $f(x) = \{f(x_1), f(x_2), \ldots\}$
- Scalar product between functions f_1 and f_2:
  - Real, unbounded domain $\int_{-\infty}^{\infty} f_1(x) f_2(x) dx$
  - Real, bounded domain $\int_{x_{\min}}^{x_{\max}} f_1(x) f_2(x) dx$
  - Integer, unbounded domain $\sum_{n=-\infty}^{\infty} f_1(n) f_2(n)$
  - Integer, bounded domain $\sum_{n=0}^{N-1} f_1(n) f_2(n)$
- Orthogonal bases can be defined and used as for vectors

Function as Vector

- An image can be understood as a function with a finite domain
- Number of image elements = number of function values = vector dimension



Representation of image row (of image function)

Image row (quite short!):  corresponds to (30, 211)

i.e. $(30, 211) = 30 \cdot (1,0) + 211 \cdot (0,1)$

Vectors of orthogonal basis

Transformation into another basis

The transformation into another basis **is invertible if**

- it is **orthogonal**
- the **number of basis vectors remains the same**

Transformation into Vector-Matrix Notation

1. Function can be represented als Vektor $\vec{f}$ of its function values

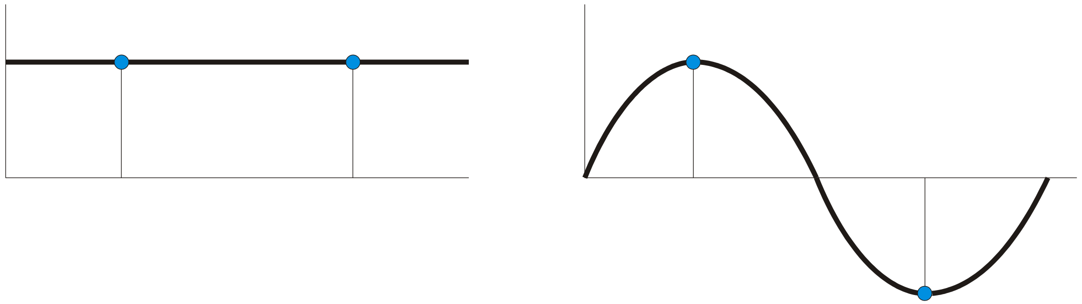
2. Basis functions b_u can be represented as a quadratic matrix **B** :

$$\mathbf{B} = \begin{pmatrix} b_0(0) & b_1(0) & \dots & b_{N-1}(0) \\ b_0(1) & b_1(1) & & \dots \\ \dots & & & \\ b_0(N-1) & \dots & & b_{N-1}(N-1) \end{pmatrix}$$

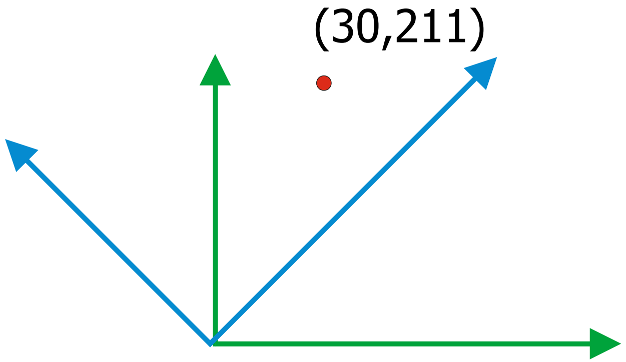
3. Transformation onto new basis *F* if multiplication of vector with matrix **B**:

$$\vec{F} = \vec{f} \times \mathbf{B}$$

Orthogonal Periodic Fuctions



Basis:
(1, 1)
(1, -1)



Transform:

$$(30,211) * \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = (241, -181)$$

Inverse transform:

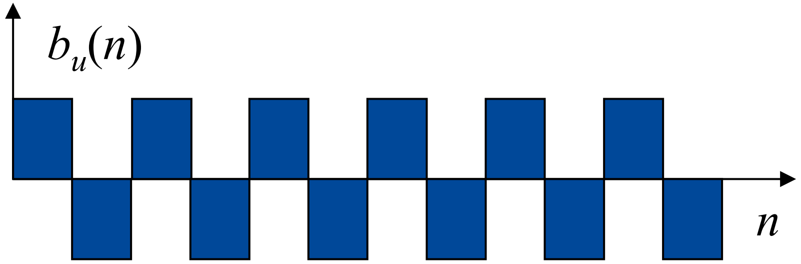
$$(241,-181) * \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}^T = (60, 422)$$

Comment: Result of inverse transform has to be scaled, as the basis is not normalised

Complete Basis

- Let basis functions be cosine function with frequencies 0,1,2,…:

  $b_u(n) = \cos(nu \cdot 2\pi / N)$, u=0,1,2…
- Problem: The maximum number of cycles that can be represented for a function with N values is N/2 -> Number of basis functions is N/2.

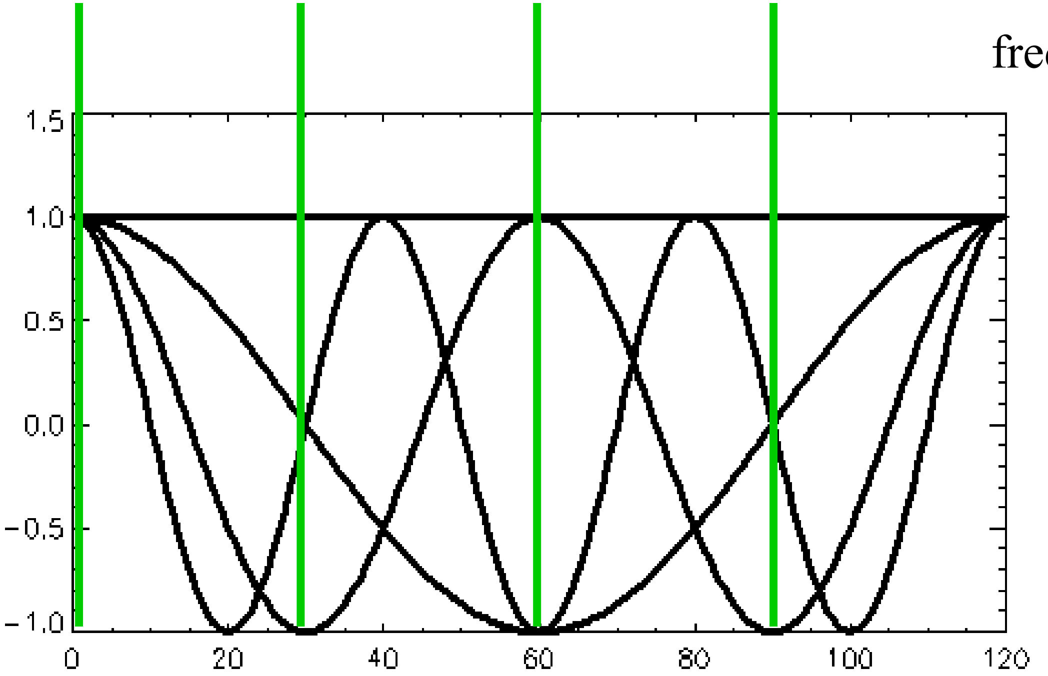


- Additionally required basic functions should have the same semantics

Example: Four Basis Functions

sampling intervals

frequencies



Sampled cosine values

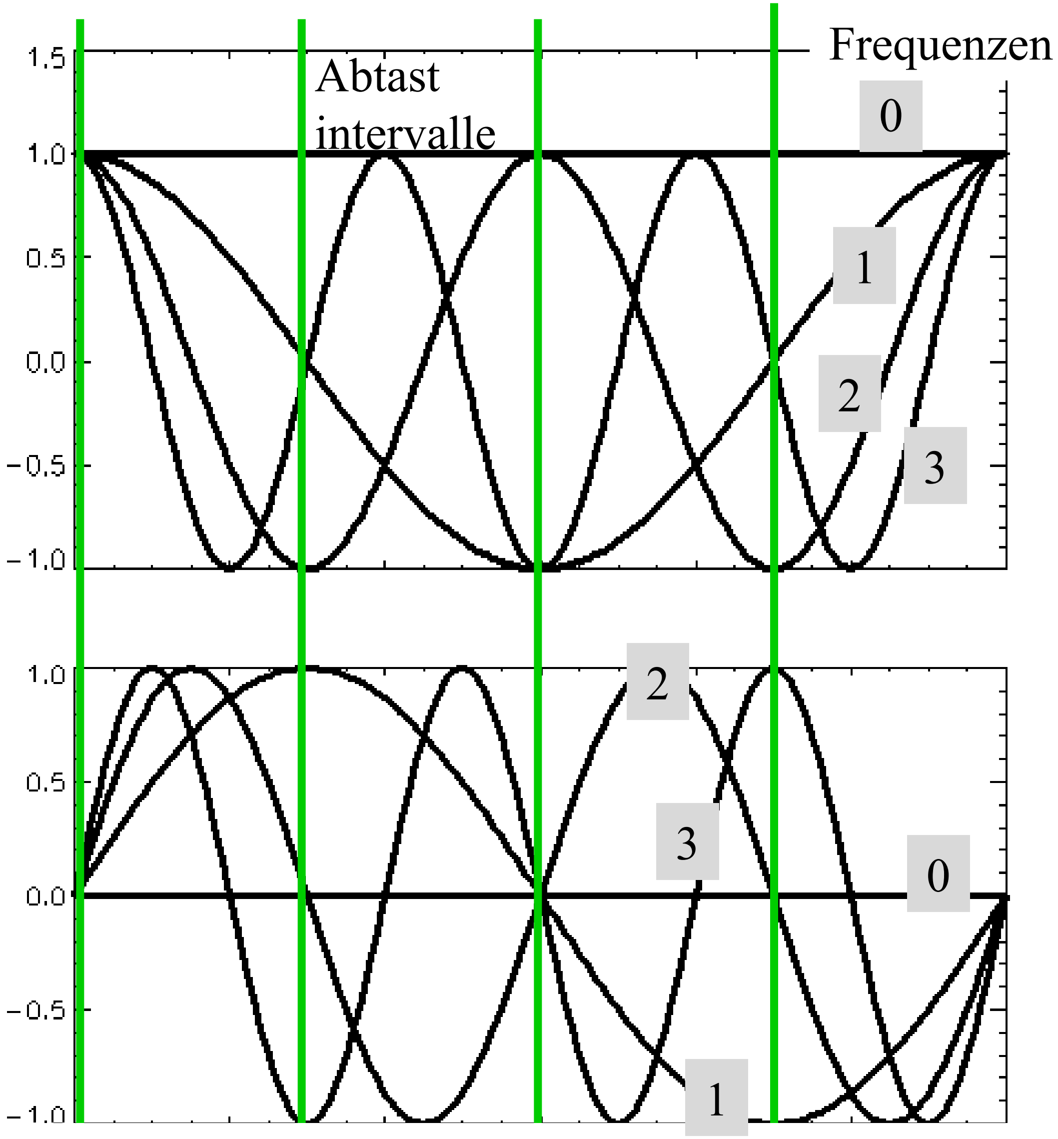
(1   1   1   1)

(1   0 –1   0)

(1 –1   1 –1)

(1   0 –1   0)

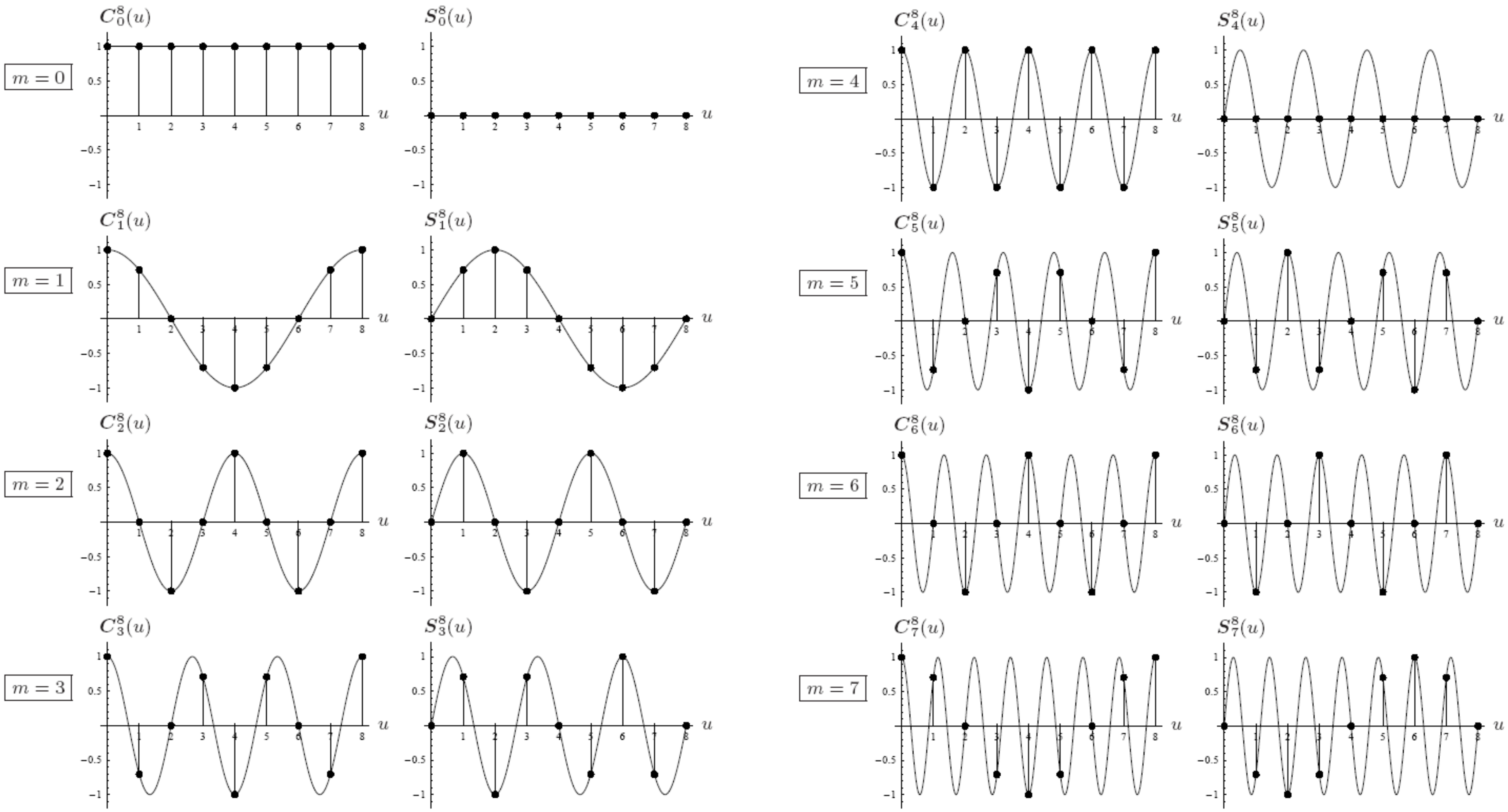
Basis Function Pairs



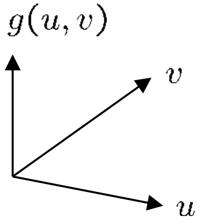
Sampled cosine values:
(1 1 1 1)
(1 0 –1 0)
(1 –1 1 –1)
(1 0 –1 0)

Sampled sine values
(0 0 0 0)
(0 1 0 –1)
(0 0 0 0)
(0 –1 0 1)

Discrete Basis Functions (for m=8)

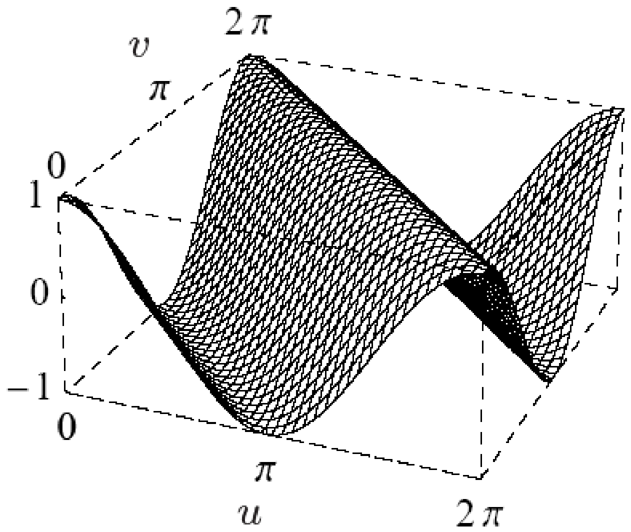


Fourier Transform in 2D

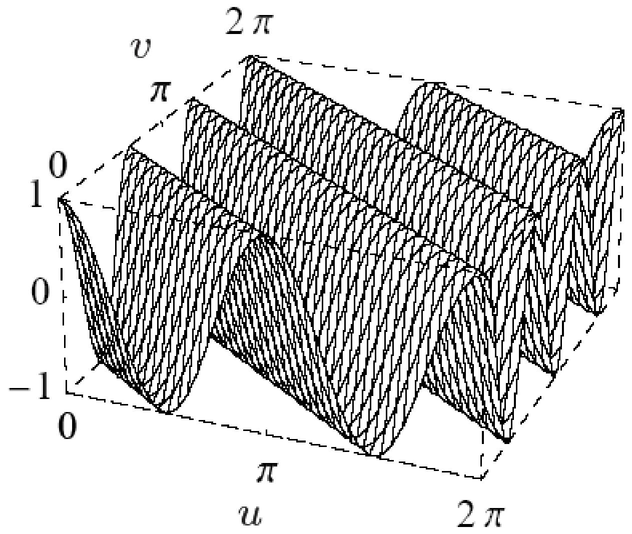


$$\boxed{\begin{aligned} C(u,v) &= \cos(\omega_x u + \omega_y v) \\ S(u,v) &= \sin(\omega_x u + \omega_y v) \end{aligned}}$$

ω_x, ω_y: Frequencies in x and y direction

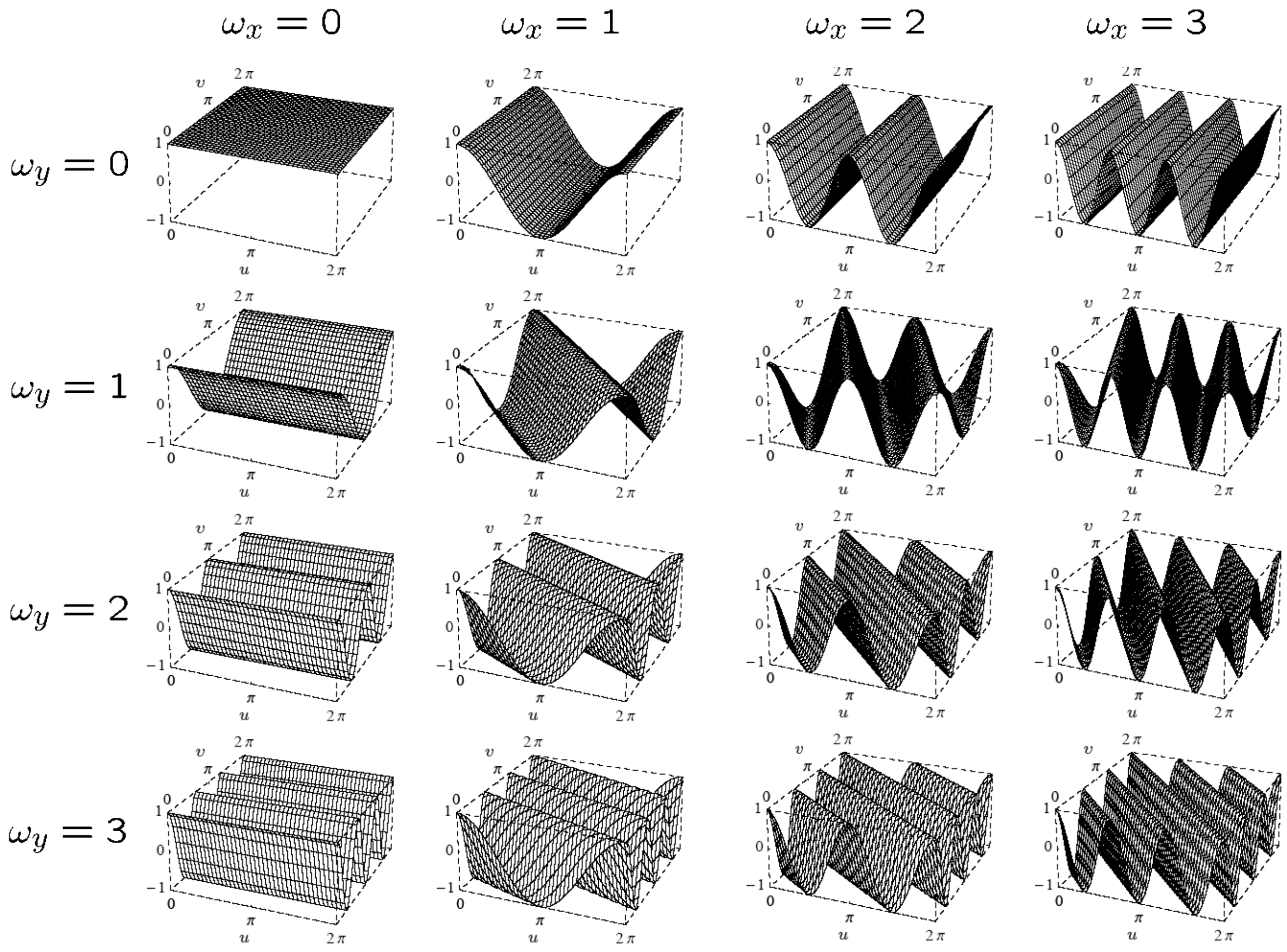


$\omega_x = \omega_y = 1$



$\omega_x = 2,\ \omega_y = 3$

Examples for 2D Basis Functions



By combining different horizontal and vertical frequencies, we can generate sine and cosine waves of any direction and frequencies can be generated

Representation as Exponential Function

Taylor series expansion for cosine and sine:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots \quad \sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

Taylor series expansion for e^{ix}:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots \Rightarrow e^{ix} = 1 + \frac{ix}{1!} + \frac{(ix)^2}{2!} + \frac{(ix)^3}{3!} + \dots$$

Therefore, it holds because of $i^2 = -1$: $\cos(x)+i\cdot\sin(x) = e^{ix}$

Phase shift a can be expressed as multiplication in complex functions:

$$\cos(x+\alpha)+i\cdot\sin(x+\alpha)=e^{i(x+\alpha)}=e^{i\alpha}\cdot e^{ix}$$

1D Basis Function

Imag function: $f(n)$, n=0,N-1,

i.e.: N basis functions $b_u(n) = e^{i \cdot 2\pi \cdot \left(\frac{u \cdot n}{N}\right)}$ with frequencies u=0 to N-1

e.g. $b_0(n)$ = [(1,1), (1,1), ... , (1,1)]

Transform FT: FT(f) = F = f · B (vector-matrix notation)

$$F(u) = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} f(n) \cdot e^{-i \cdot 2\pi \frac{u \cdot n}{N}}$$

for all u=0 to N-1

Inverse transform FT^{-1} : $FT^{-1}(F) = F \cdot B^T$ (vector-matrix notation)

$$f(n) = \frac{1}{\sqrt{N}} \sum_{u=0}^{N-1} F(u) \cdot e^{i \cdot 2\pi \frac{u \cdot n}{N}}$$

für alle n=0,N-1

Scaling factor, as basis functions are not normalised

2D Transformation Pair

$$F(u,v)=\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)\cdot e^{-i\cdot 2\pi\cdot\left(\frac{um}{M}+\frac{vn}{N}\right)}$$

$$f(m,n)=\frac{1}{MN}\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} F(u,v)\cdot e^{i\cdot 2\pi\cdot\left(\frac{um}{M}+\frac{vn}{N}\right)}$$

Transformation pair for images of size *M*x*N*

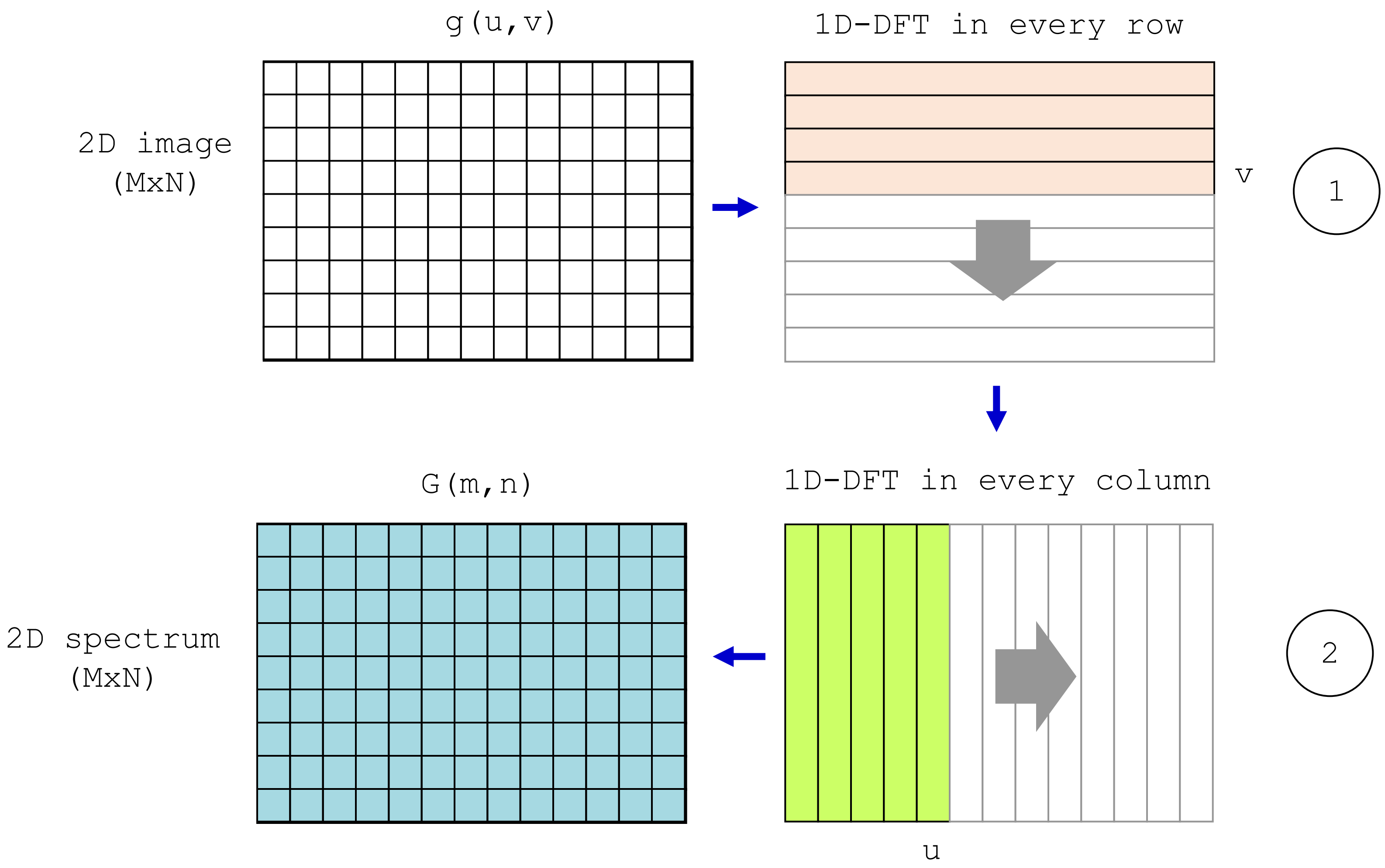
Implementation of 2D Transform

$$\boxed{G(m,n) = \frac{1}{\sqrt{MN}} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} g(u,v) \cdot e^{-\mathrm{i}2\pi \frac{mu}{M}} \cdot e^{-\mathrm{i}2\pi \frac{nv}{N}}}$$

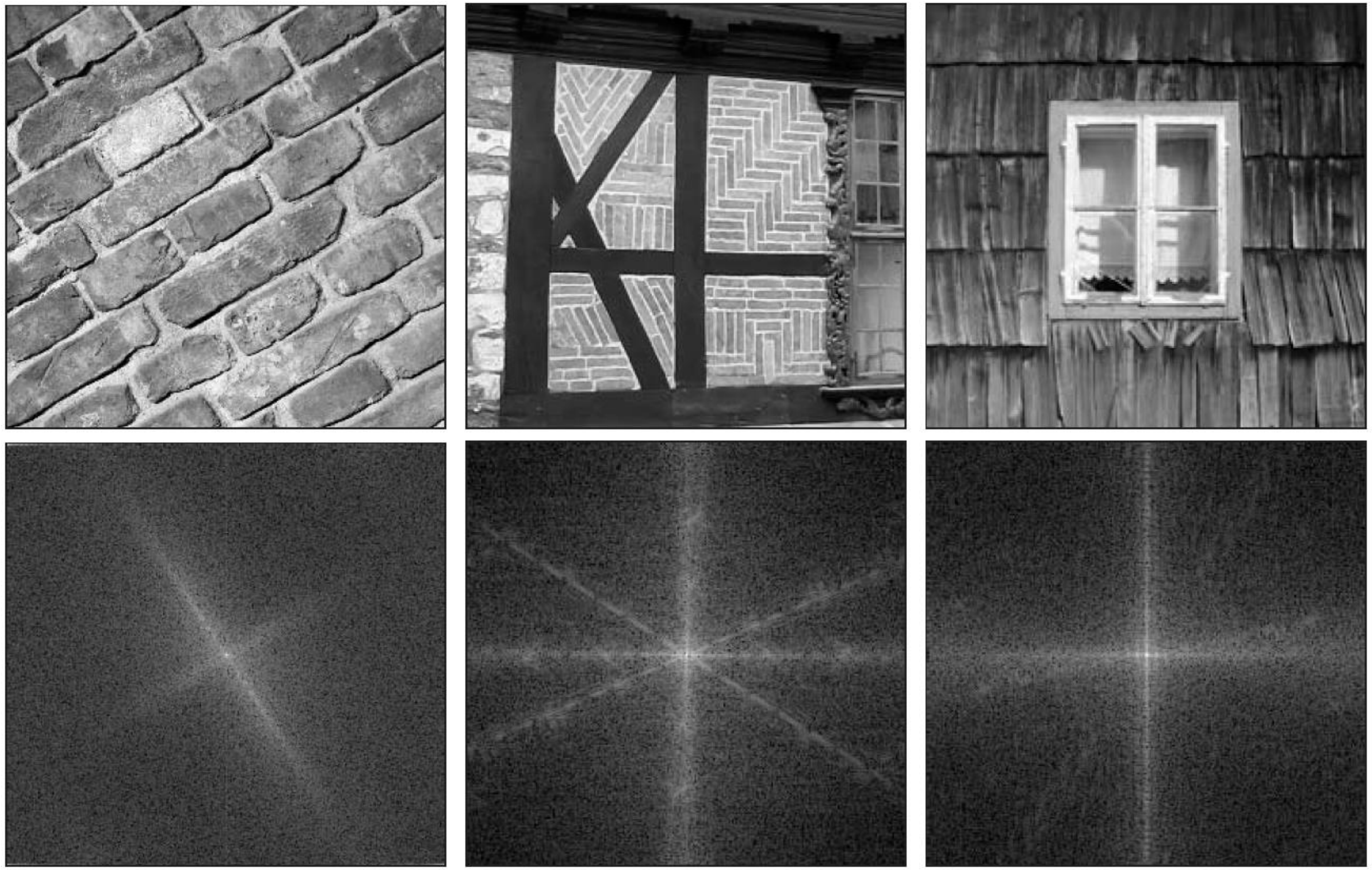
$$G(m,n) = \frac{1}{\sqrt{N}} \sum_{v=0}^{N-1} \underbrace{\left[\frac{1}{\sqrt{M}} \sum_{u=0}^{M-1} g(u,v) \cdot e^{-\mathrm{i}2\pi \frac{mu}{M}} \right]}_{\text{1-dim. DFT of row } g(\cdot,v)} \cdot \; e^{-\mathrm{i}2\pi \frac{nv}{N}}$$

- 2D DFT ist SEPARABLE in x and y direction!
- 1D DFT (bzw. FFT) is sufficient for 2D transform
- Computation can be done “in place”

In-Place Computation of DFT



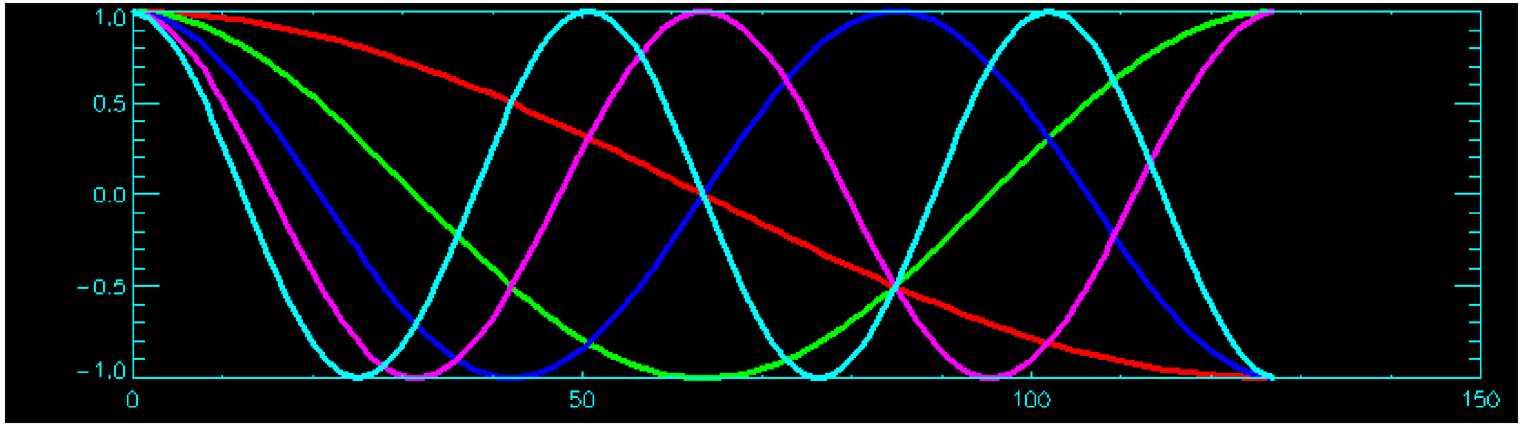
2D Spectra (Examples)

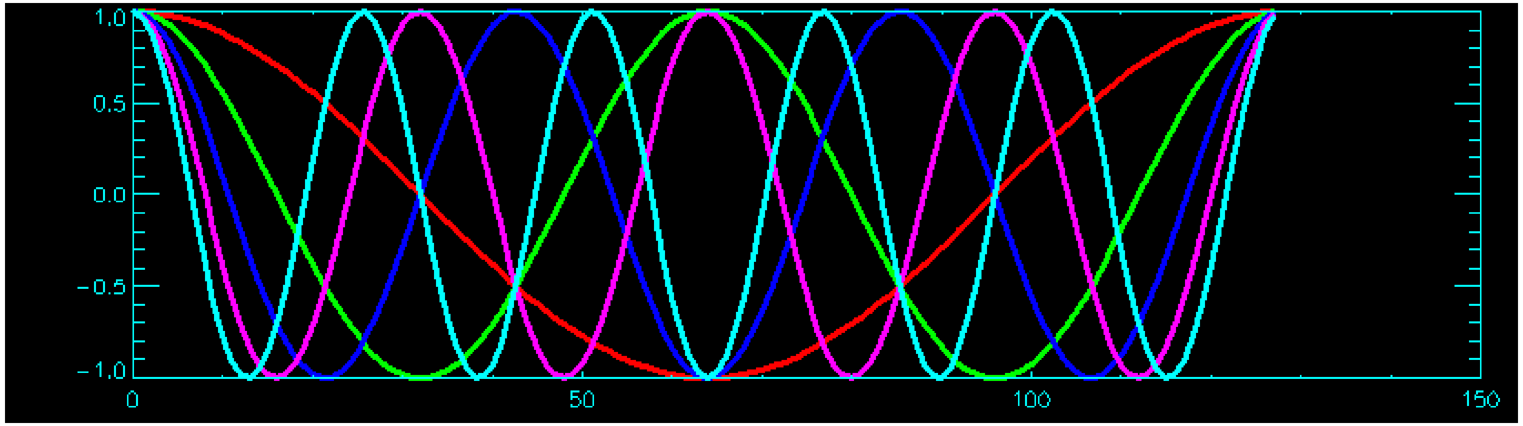


Signal

Spectrum

Basis Functions of DCT / DFT





Source: K. Tönnies, Lecture material „Grundlagen der Bildverarbeitung", 2005

Basis Functions of Discrete Cosine Transform

$$C(u,v) = \alpha(u)\alpha(v)\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)\cos\left(\frac{(2m+1)u\pi}{2M}\right)\cos\left(\frac{(2n+1)v\pi}{2N}\right)$$

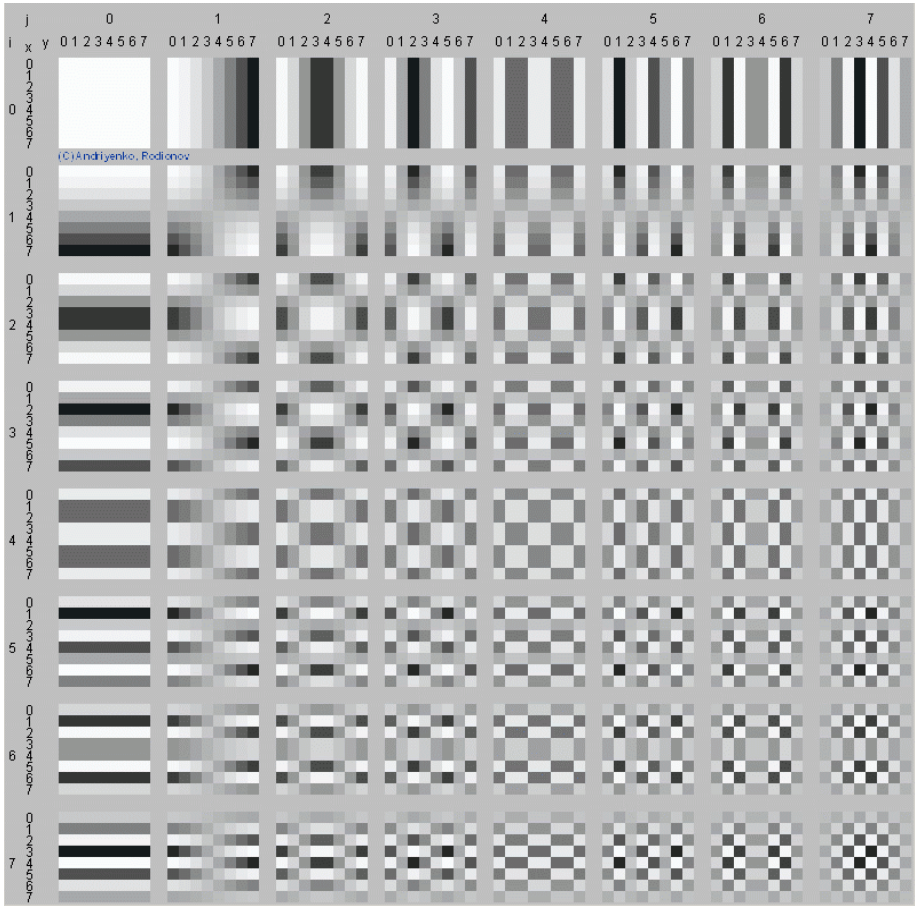
$$f(m,n) = \sum_{u=0}^{M-1}\sum_{v=0}^{N-1}\alpha(u)\alpha(v)C(u,v)\cos\left(\frac{(2u+1)m\pi}{2M}\right)\cos\left(\frac{(2v+1)n\pi}{2N}\right).$$

$$\alpha(u) = \begin{cases}\sqrt{1/M} & \text{, für } u = 0\\ \sqrt{2/M} & \text{, für } u \neq 0\end{cases} \quad \text{und} \quad \alpha(v) = \begin{cases}\sqrt{1/N} & \text{, für } v = 0\\ \sqrt{2/N} & \text{, für } v \neq 0\end{cases}$$

Basis functions are waves
Their values have different signs for every second frequency u
at the beginning and at the end.

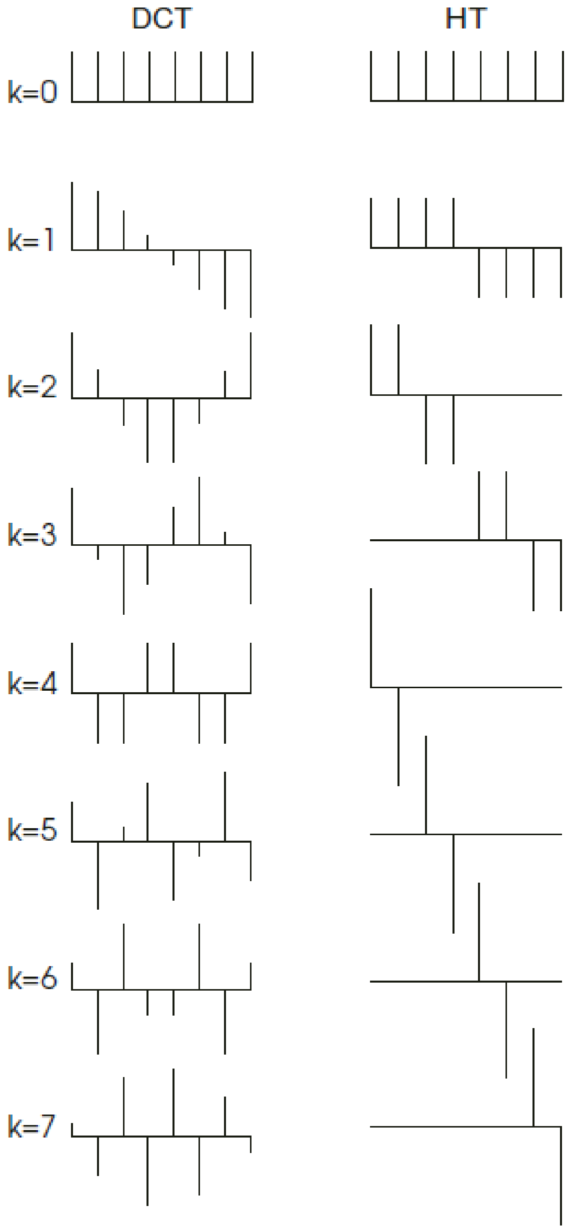
DCT: Example

- Basis functions for blocks of size 8x8



Basis Functions for DCT & Discrete Wavelet Transform

- Comparison of basis functions for
  - DCT and
  - Haar Wavelet
- Eight (8) data points



Source: T. Strutz, „Bilddatenkompression", 2009

Wavelet(s)

- A wavelet (”Wellchen“) is a function f(x), where f(x) != 0 only in a compact interval.

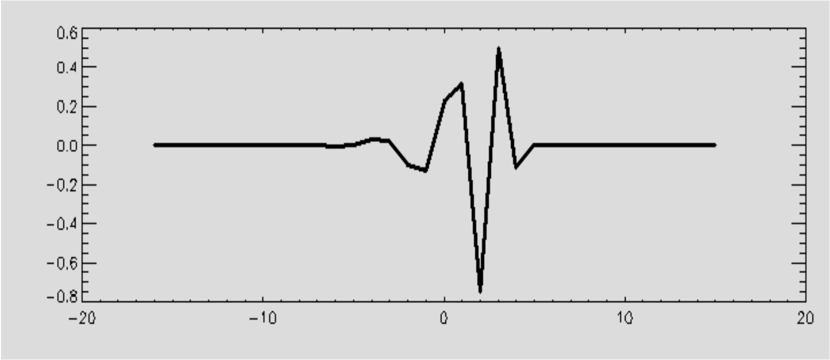
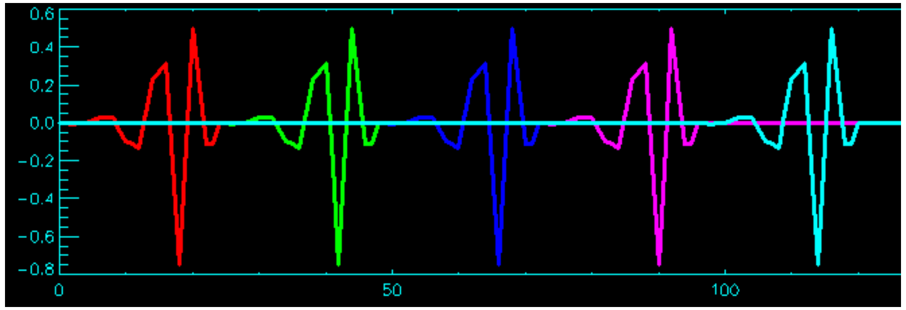
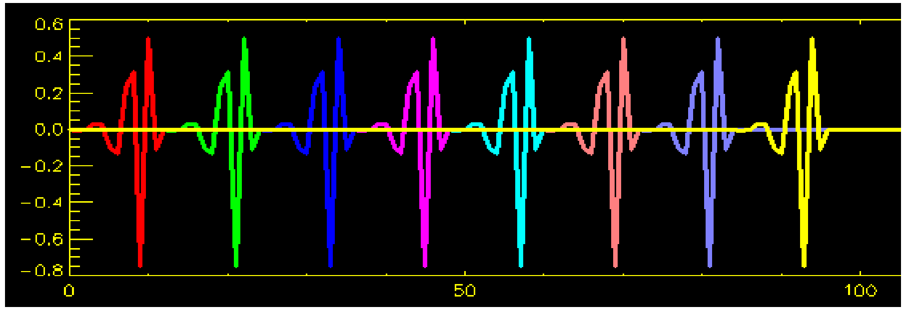


Image source: K. Tönnies,
Lecture material „Grundlagen der Bildverarbeitung“, 2005

Wavelets: Basis Functions

- Wavelet basis functions are defined by their
  - **location** (generated by **translation** of the mother wavelet)
  - **frequency** (generated by **scaling** the mother wavelet)





Source: K. Tönnies, Lecture material „Grundlagen der Bildverarbeitung", 2005

Computation of the Wavelet Transform

- Basically it can be computed by projection on the basis functions
- Scaling and shifting can be used to compute DWT efficiently
  - Scaling : Generate input values for level s via scaling
  - Shifting : Compute coefficients of a scale level via convolution

Wavelets, Filters and Filter Banks

- Implementing Discrete Wavelet Transform with filter banks

- Filterbank consists of
  - Low-pass filter (“moving average“)
  - High-pass filter (“moving difference“)

- A **filter** is a linear operator h for input vector x

$$y(t) = \sum_k h(k)x(t-k)$$

That is: **y** is the convolution of **x** with a fixed vector **h**;
h represents the filter coefficients

A Simple Example for a Filter

- Given the signal x=(7, 6, 7, 1, 1, 0, 1, 8, 7)
- Low-pass filter (“moving average“)

  Very simple low-pass filter h_o
  - $h_o(0)$ = ½
  - $h_o(1)$ = ½
- High-pass filter („moving difference“)

  Very simple high-pass filter h_1
  - $h_1(0)$ = ½
  - $h_1(1)$ = -½
- Result
  - `y =(6.5, 6.5, 4.0, 1.0, 0.5, 0.5, 4.5, 7.5, 3.5)`
  - `y‘=(0.5,-0.5, 3.0, 0.0, 0.5,-0.5,-3.5, 0.5, 3.5)`

Filters, Filter Banks, and Wavelets

- A filter bank consists of a set of filters

- Analysis filter bank
    - Decomposes signal into frequency bands
    - Has often two filters: low-pass and high-pass
    - More filters are possible

- Synthesis filter bank
    - Re-combines frequency bands
    - That is, the signal can be reconstructed (important for compression, for example)

Filter Bank with Low-Pass and High-Pass Filter

- Using only low-pass or high-pass results: reconstruction not possible
  - But „together they are strong“ ;)

- Example signal can be reconstructed with y and y‘
  - `y =(6.5, 6.5, 4.0, 1.0, 0.5, 0.5, 4.5, 7.5, 3.5)`
  - `y‘=(0.5,-0.5, 3.0, 0.0, 0.5,-0.5,-3.5, 0.5, 3.5)`

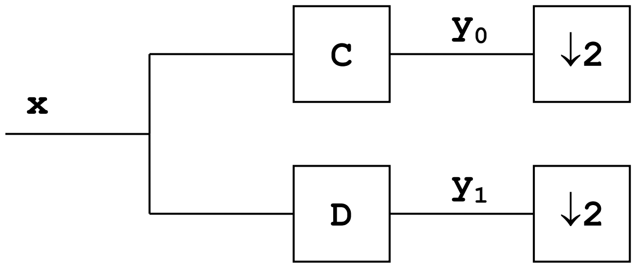
- But bad news!? Required memory space has doubled…

Filter Bank with Low-Pass and High-Pass Filter

- Good news:
  - y and y‘ can be „decimated“ (downsampling) by factor 2
  - signal x can still be reconstructed

- $(\downarrow 2)y = (\ldots, y(-4), y(-2), y(0), y(2), y(4), \ldots)$

- Block diagram of a two-channel filter bank



Filter Bank with Low-Pass and High-Pass Filter

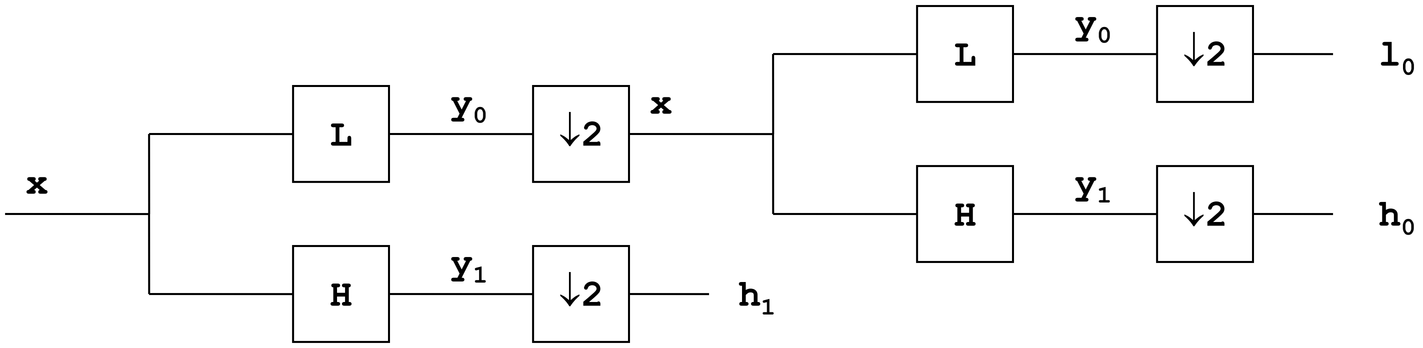
- Reconstruction using
  - `y =(6.5, 6.5, 4.0, 1.0, 0.5, 0.5, 4.5, 7.5, 3.5)`
  - `y‘=(0.5,-0.5, 3.0, 0.0, 0.5,-0.5,-3.5, 0.5, 3.5)`

- Reconstruction with half of (doubled) data possible
  - y =(**6.5**, 6.5, **4.0**, 1.0, **0.5**, 0.5, **4.5**, 7.5, **3.5**)
  - y‘=(**0.5**,-0.5, **3.0**, 0.0, **0.5**,-0.5,**-3.5**, 0.5, **3.5**)

  => ↓2
  - y =(**6.5, 4.0, 0.5, 4.5, 3.5**)
  - y‘=(**0.5, 3.0, 0.5,-3.5, 3.5**)

Multiresolution

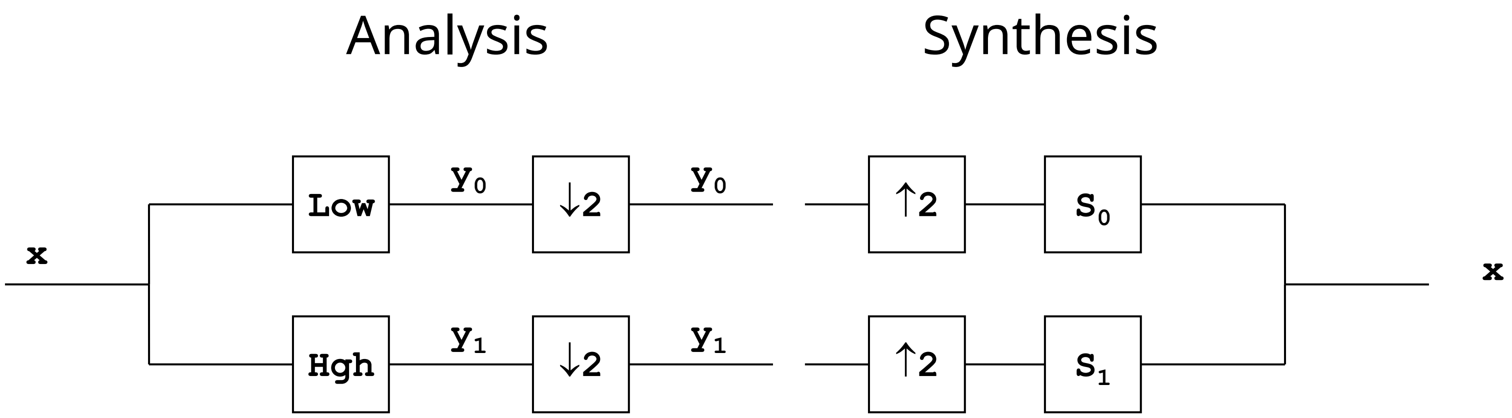
- Iterative application of low-pass Filter and downsampling
- Tree-like structured filter bank



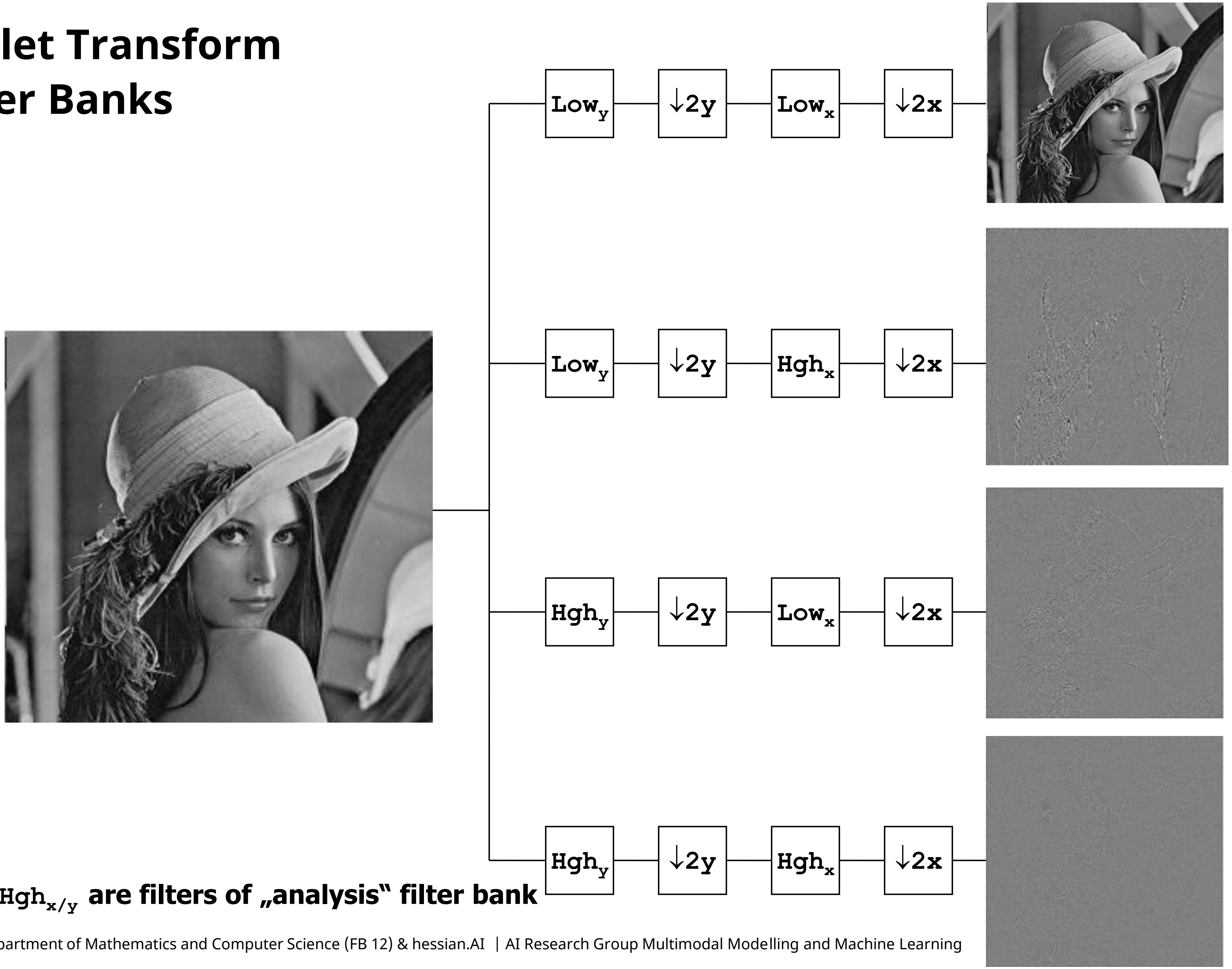
- Three Wavelet coefficients:
  - l_0: “average“ at ½ temporal resolution
  - h_1 : difference at full temporal resolution
  - h_o: difference at ½ temporal resolution

Filter Bank: Analysis and Synthesis

- Representation of two-channel filter bank
- Decomposition (analysis) and reconstruction (synthesis) of signal

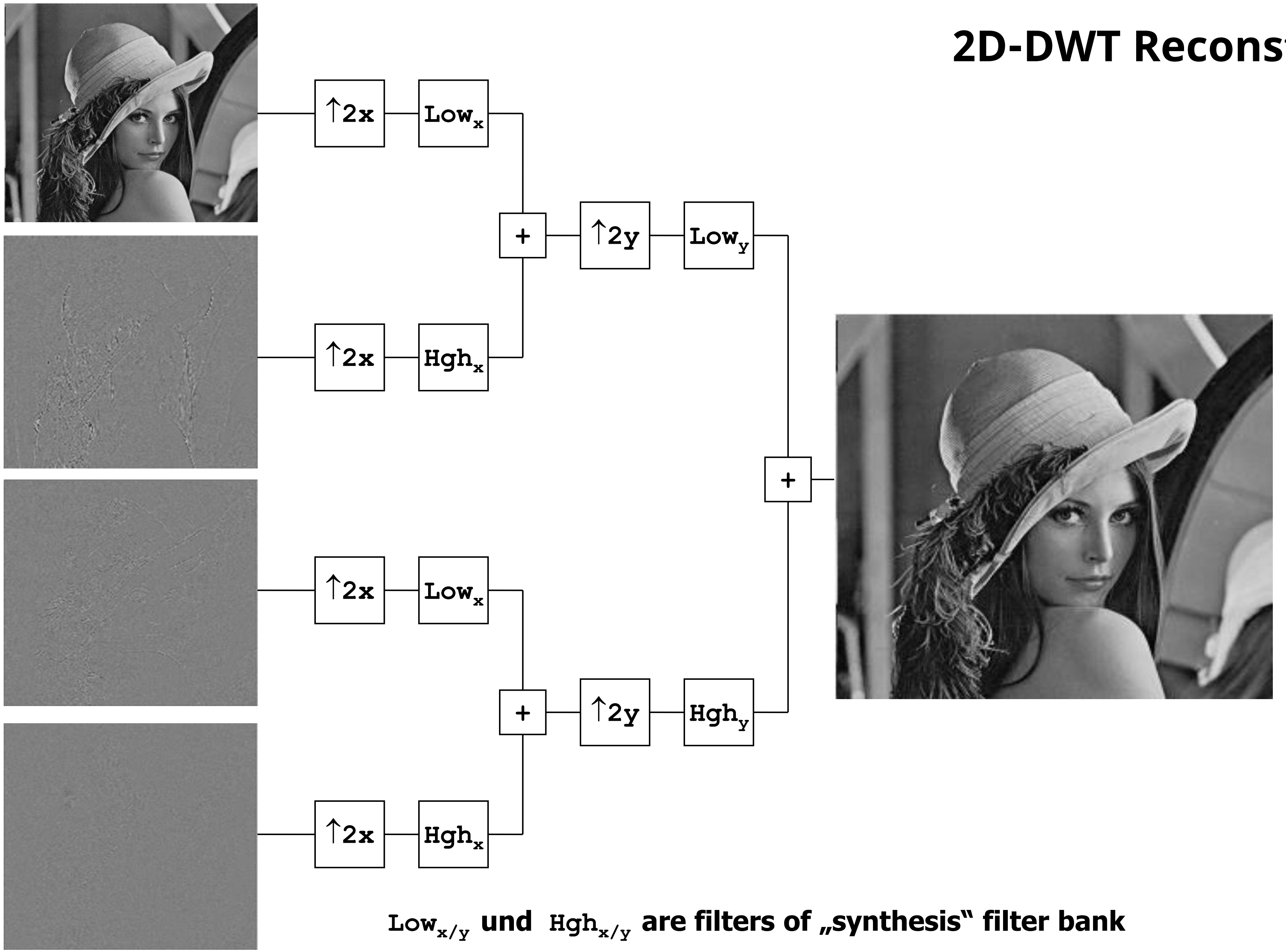


2D Wavelet Transform with Filter Banks



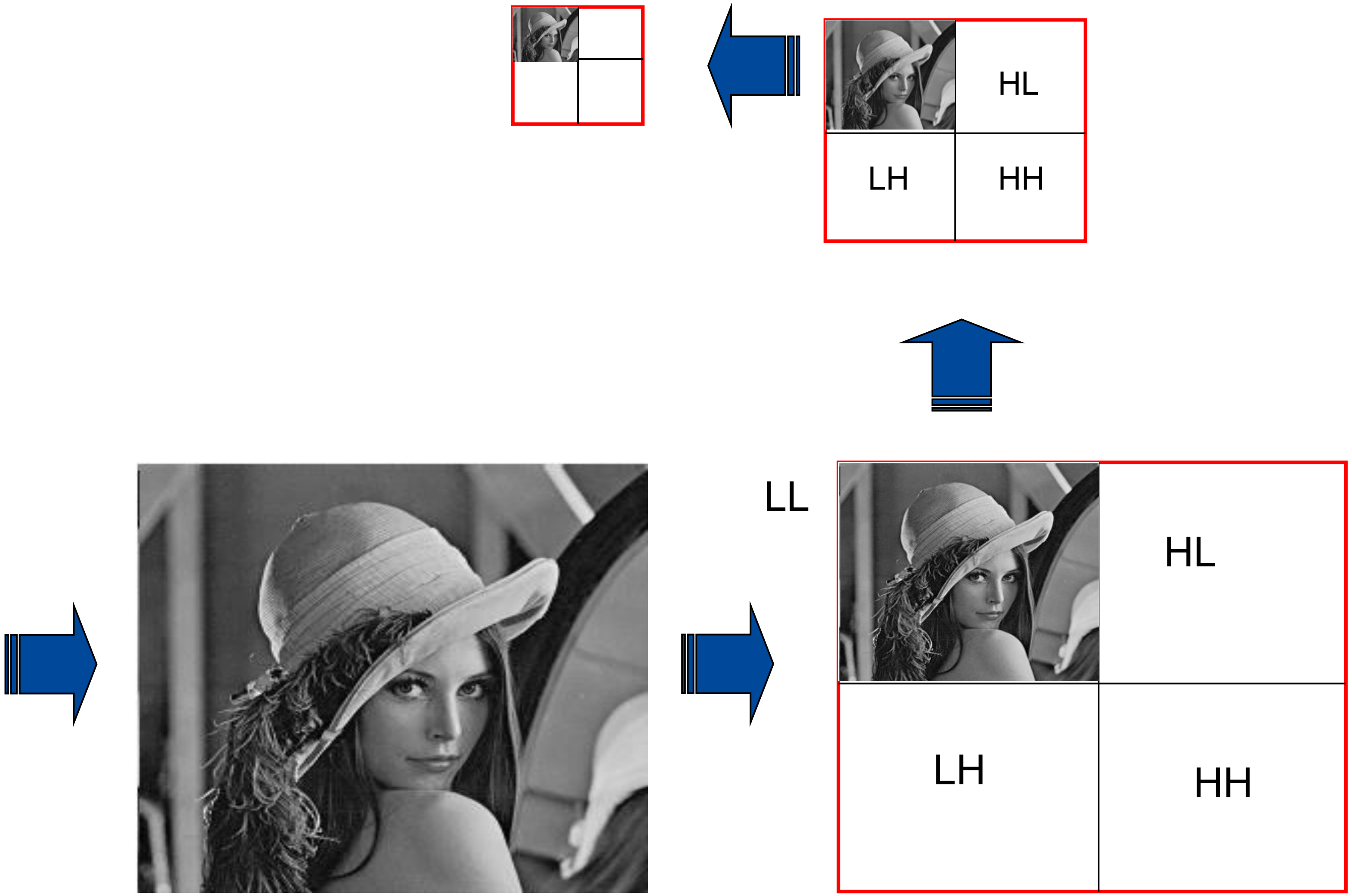
`Low`$_{x/y}$ and `Hgh`$_{x/y}$ are filters of „analysis" filter bank

2D-DWT Reconstruction

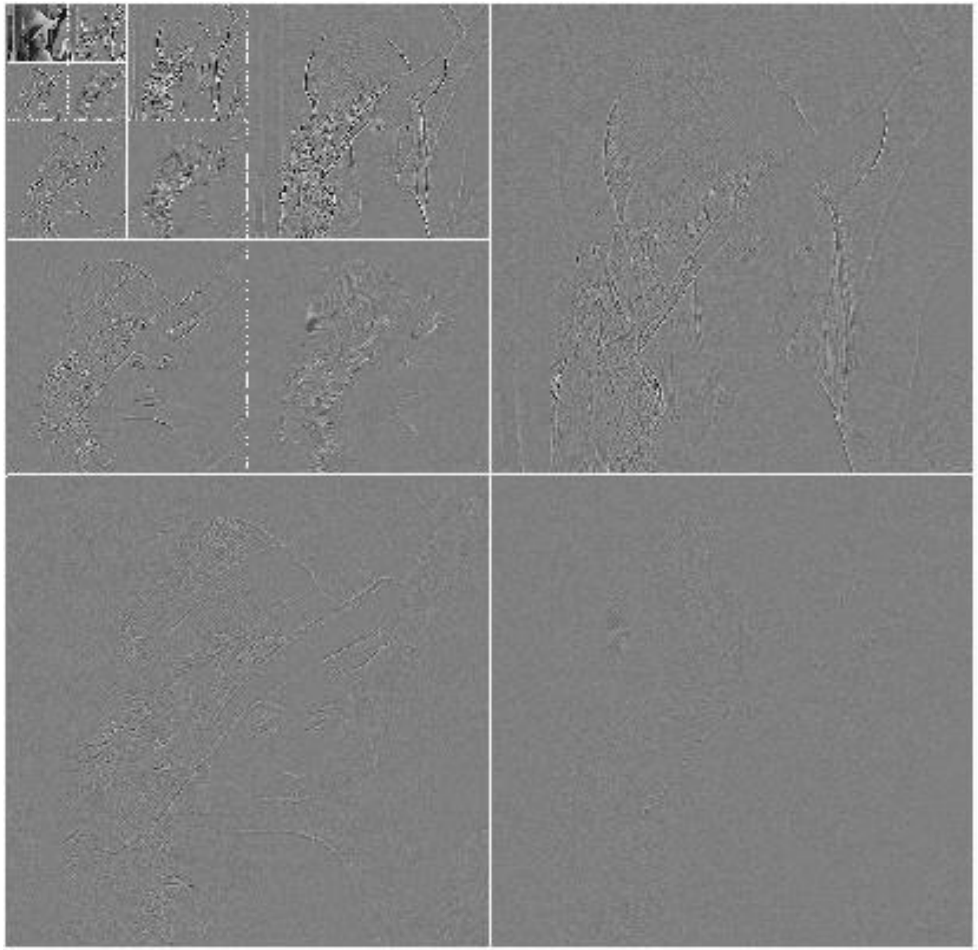


$\mathtt{Low}_{x/y}$ **und** $\mathtt{Hgh}_{x/y}$ **are filters of „synthesis" filter bank**

Iterative Application of 2D Wavelet Transformation (1)

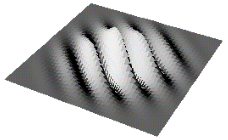


Iterative Application of 2D Wavelet Transformation (2)

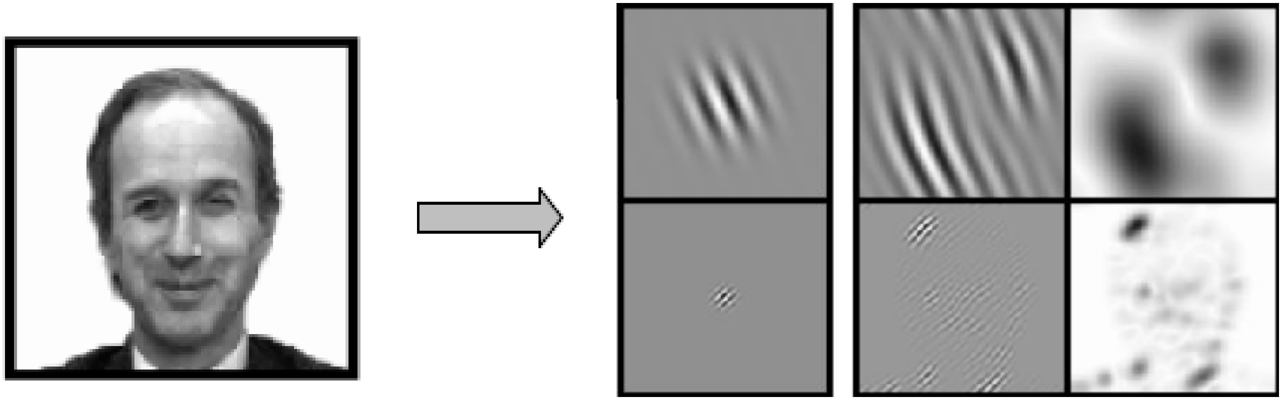


Source image:
Wiegand/Girod

Gabor Wavelet Transform

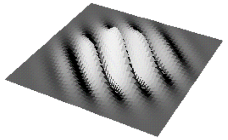


- Biological motivaton based on principle of human visual system
- Tolerant with respect to brightness changes in an image
- Edge detection at
  - different orientations
  - different scales



Source of images: Wiskott et al. 1999

Gabor Wavelets



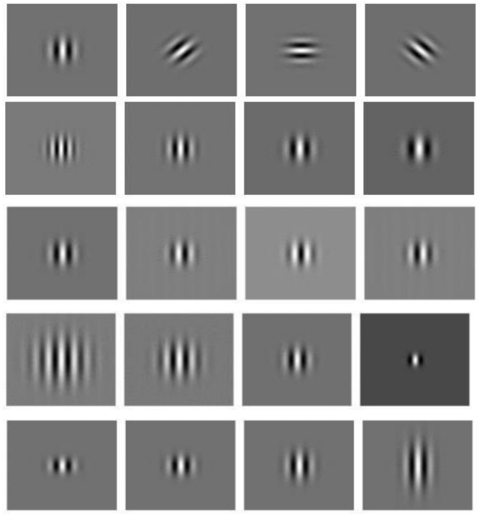
- Gabor wavelets are controlled via 5 parameters
  - Orientation θ
  - Wave length λ (~frequency)
  - Phase φ
  - Radius of Gaussian function σ
  - Aspect ratio γ

- Construction of g:

$$g_{\theta,\lambda,\varphi,\sigma,\gamma}(x,y) = e^{-\frac{x'^2+\gamma^2 y'^2}{2\sigma^2}} \cos(2\pi \frac{x'}{\lambda} + \varphi)$$

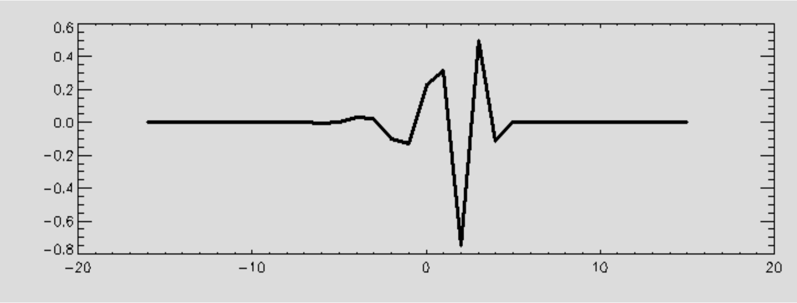
$$x' = x\cos\theta + y\cos\theta$$

$$y' = -x\sin\theta + y\cos\theta$$



Scale and Shift

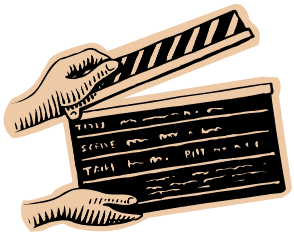
- Wavelet basis functions Y
  - are determined by location (time) d and
- –
  - scaling level s : $\Psi_{s,d}$
- For basis functions Y and scaling s and location d (shift in time) applies:
  - $\Psi_{s,0}(n) = \Psi_{s+1,0}(2n)$ (scaling feature, s-1 lower frequency)
  - $\Psi_{s,d}(n) = \Psi_{s,0}(n + d \cdot \mathbf{2}^{s})$ (shift in time / translation)



Lessons Learned

- Image representation/descriptor via
  - Fourier transform
  - Discrete Cosine Transform
  - Wavelet transform
  - Wavelet transform and filter banks

Questions and Comments...?



Thanks for your attention!